

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler & Dasar Pemrograman

Yusuf Fadilah Pradana Utama - 5024241025

2026

1 Langkah-Langkah Percobaan

Langkah-langkah percobaan pada Modul 1 dilakukan melalui lima tahapan utama dengan menggunakan board STM32 Blackpill dan ESP32 DevKit. Setiap percobaan dilakukan secara berurutan sesuai dengan instruksi pada modul. Adapun langkah-langkah yang dilakukan adalah sebagai berikut.

1. **Percobaan 1: Pengenalan STM32 Blackpill (Blink)**

   (a) Menyiapkan board STM32 Blackpill, ST-Link V2, kabel jumper female-female, serta laptop atau PC yang telah terpasang Visual Studio Code dan ekstensi PlatformIO.

   (b) Membuka project PlatformIO Modul 1 yang telah dibuat sebelumnya.

   (c) Mengatur konfigurasi pada file `platformio.ini` dengan memilih platform STM32, board `blackpill_f401cc`, framework Arduino, dan upload protocol ST-Link.

   (d) Menghubungkan ST-Link dengan board STM32 Blackpill menggunakan kabel jumper. Pin SWDIO dihubungkan ke SWDIO, SWCLK ke SWCLK, GND ke GND, dan 3.3V ke 3V3.

   (e) Memastikan ST-Link menggunakan tegangan 3.3V sebelum dihubungkan ke komputer melalui kabel USB.

   (f) Membuka file `src/main.cpp`, kemudian menuliskan program Blink menggunakan pin PC13 sebagai output LED onboard.

   (g) Melakukan proses Build untuk memeriksa apakah program dapat dikompilasi tanpa kesalahan.

   (h) Setelah proses Build berhasil, melakukan Upload untuk mengunggah program ke board STM32 melalui ST-Link.

   (i) Mengamati LED onboard pada pin PC13. LED akan menyala selama 500 ms dan mati selama 500 ms secara berulang.

2. **Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)**

   (a) Menyiapkan board ESP32 DevKit dan kabel USB untuk menghubungkan board dengan komputer.

   (b) Membuka kembali project PlatformIO Modul 1 yang sama dengan percobaan sebelumnya.

   (c) Mengubah konfigurasi pada file `platformio.ini` dengan memilih platform `espressif32`, board `esp32dev`, dan framework `espidf`.

   (d) Menunggu proses refresh atau re-index konfigurasi PlatformIO hingga selesai sebelum melanjutkan proses pemrograman.

   (e) Menghubungkan ESP32 ke komputer menggunakan kabel USB dan memastikan komputer dapat mengenali port komunikasi board.

   (f) Menghapus file `src/main.cpp`, kemudian membuat file baru bernama `src/main.c` karena program menggunakan framework ESP-IDF.

   (g) Menuliskan program Blink dengan menggunakan fungsi GPIO dari ESP-IDF. Pin GPIO 2 digunakan sebagai output untuk mengendalikan LED onboard.

   (h) Melakukan proses Build untuk memastikan program berhasil dikompilasi.

(i) Melakukan Upload untuk mengunggah program ke board ESP32.

(j) Mengamati LED onboard ESP32 yang menyala dan mati secara bergantian dengan interval 500 ms.

(k) Mengamati struktur program dan fungsi yang digunakan, kemudian membandingkannya dengan program Blink pada percobaan pertama yang menggunakan framework Arduino.

3. **Percobaan 3: Pull-up dan Pull-down Eksternal vs Internal**

(a) Menyiapkan board ESP32 DevKit, breadboard, dua pushbutton, tiga resistor 10kΩ, kabel jumper, dan kabel USB.

(b) Mengubah konfigurasi `platformio.ini` menggunakan platform `espressif32`, board `esp32dev`, dan framework Arduino.

(c) Menghapus file `src/main.c`, kemudian membuat kembali file `src/main.cpp` untuk menyesuaikan penggunaan framework Arduino.

(d) Memasang dua pushbutton pada breadboard dan menghubungkannya dengan ESP32 sesuai skema rangkaian pull-up dan pull-down eksternal.

(e) Menghubungkan pushbutton pertama ke GPIO 32 dengan konfigurasi pull-up eksternal. Salah satu kaki tombol dihubungkan ke GPIO 32 dan 3.3V melalui resistor 10kΩ, sedangkan kaki lainnya dihubungkan ke GND.

(f) Menghubungkan pushbutton kedua ke GPIO 33 dengan konfigurasi pull-down eksternal. Salah satu kaki tombol dihubungkan ke GPIO 33 dan GND melalui resistor 10kΩ, sedangkan kaki lainnya dihubungkan ke 3.3V.

(g) Memastikan seluruh sambungan rangkaian telah sesuai sebelum menghubungkan ESP32 ke komputer.

(h) Menuliskan Kode Program A pada file `src/main.cpp`. Program menggunakan konfigurasi `INPUT` untuk membaca kondisi kedua tombol karena resistor pull-up dan pull-down telah dipasang secara eksternal.

(i) Melakukan Build dan Upload program ke ESP32.

(j) Membuka Serial Monitor dengan baud rate 115200 untuk mengamati kondisi kedua pushbutton.

(k) Menekan dan melepaskan masing-masing tombol, kemudian mencatat perubahan kondisi yang ditampilkan pada Serial Monitor.

(l) Setelah pengujian eksternal selesai, melepas resistor pull-up dan pull-down dari rangkaian tanpa mengubah posisi kedua pushbutton.

(m) Mengatur kembali rangkaian untuk menggunakan pull-up dan pull-down internal. Pushbutton pertama dihubungkan antara GPIO 32 dan GND, sedangkan pushbutton kedua dihubungkan antara GPIO 33 dan 3.3V.

(n) Mengubah program menggunakan Kode Program B dengan konfigurasi `INPUT_PULLUP` pada GPIO 32 dan `INPUT_PULLDOWN` pada GPIO 33.

(o) Melakukan Build dan Upload kembali, kemudian membuka Serial Monitor untuk mengamati hasil pembacaan tombol menggunakan resistor internal.

(p) Membandingkan hasil pengujian pull-up dan pull-down eksternal dengan internal. Hasil yang diharapkan adalah kedua metode menghasilkan kondisi logika yang sama, yaitu HIGH saat tombol pull-up tidak ditekan dan LOW saat ditekan, sedangkan pull-down menghasilkan kondisi sebaliknya.

4. **Percobaan 4: Debouncing pada Input Tombol GPIO**

(a) Menyiapkan board ESP32 DevKit, satu pushbutton, breadboard, kabel jumper, dan kabel USB.

(b) Memastikan konfigurasi `platformio.ini` masih menggunakan board `esp32dev` dengan framework Arduino seperti pada percobaan sebelumnya.

(c) Memasang pushbutton pada breadboard dengan menghubungkan salah satu kaki tombol ke GPIO 25 dan kaki lainnya ke GND.

(d) Mengatur pin GPIO 25 menggunakan konfigurasi pull-up internal sehingga kondisi awal tombol berada pada logika HIGH.

(e) Membuka file `src/main.cpp`, kemudian menuliskan program debouncing yang memiliki dua penghitung, yaitu `counterNoDebounce` dan `counterWithDebounce`.

(f) Mengatur nilai awal `DEBOUNCE_DELAY` sebesar 50 ms sebagai waktu penyaringan perubahan sinyal tombol.

(g) Melakukan Build dan Upload program ke ESP32.

(h) Membuka Serial Monitor dengan baud rate 115200 untuk mengamati hasil kedua penghitung yang ditampilkan secara bersamaan.

(i) Menekan tombol satu kali dan membandingkan perubahan nilai penghitung tanpa debouncing dengan penghitung yang menggunakan debouncing.

(j) Mengulangi pengujian dengan menekan tombol menggunakan kecepatan yang berbeda, baik secara perlahan maupun cepat.

(k) Mengubah nilai `DEBOUNCE_DELAY` menjadi 10 ms, kemudian melakukan Build dan Upload kembali. Setelah itu, menekan tombol dan mengamati perubahan kedua penghitung.

(l) Mengubah nilai `DEBOUNCE_DELAY` menjadi 200 ms, kemudian melakukan Build dan Upload kembali.

(m) Menekan tombol secara berulang dengan cepat dan mengamati perbedaan hasil pembacaan dibandingkan dengan penggunaan waktu debounce sebelumnya.

(n) Membandingkan hasil pengujian dengan waktu debounce 10 ms, 50 ms, dan 200 ms untuk mengetahui pengaruh durasi penyaringan terhadap kestabilan pembacaan dan respons tombol.

5. **Percobaan 5: Akses GPIO Analog dengan Pembacaan ADC Menggunakan LDR**

(a) Menyiapkan board ESP32 DevKit, sensor LDR, resistor 10kΩ, breadboard, kabel jumper, dan kabel USB.

(b) Memastikan konfigurasi `platformio.ini` menggunakan platform `espressif32`, board `esp32dev`, dan framework Arduino seperti pada percobaan sebelumnya.

(c) Memasang LDR pada breadboard sebagai bagian dari rangkaian pembagi tegangan.

(d) Menghubungkan salah satu kaki LDR ke sumber tegangan 3.3V, sedangkan kaki lainnya dihubungkan ke GPIO 34 dan resistor 10kΩ.

(e) Menghubungkan ujung resistor 10kΩ lainnya ke GND sehingga titik pertemuan antara LDR dan resistor menjadi masukan analog pada GPIO 34.

(f) Membuka file `src/main.cpp`, kemudian menuliskan program pembacaan ADC menggunakan fungsi `analogRead()` pada pin GPIO 34.

(g) Menambahkan perhitungan konversi nilai ADC menjadi tegangan menggunakan rumus nilai ADC dikalikan 3.3 dan dibagi 4095.

(h) Melakukan Build untuk memeriksa program, kemudian melakukan Upload untuk mengunggah program ke ESP32.

(i) Membuka Serial Monitor dengan baud rate 115200 untuk melihat nilai ADC mentah dan hasil konversi tegangan yang ditampilkan setiap 300 ms.

(j) Menutup permukaan sensor LDR menggunakan tangan untuk mengurangi cahaya yang diterima sensor, kemudian mengamati perubahan nilai ADC dan tegangan.

(k) Memberikan cahaya pada sensor menggunakan lampu atau senter, kemudian mengamati kembali perubahan nilai pembacaan pada Serial Monitor.

(l) Membandingkan nilai ADC yang diperoleh pada kondisi minim cahaya dan kondisi terang untuk mengetahui pengaruh intensitas cahaya terhadap perubahan resistansi LDR dan tegangan pada rangkaian.

2 Analisis Hasil Percobaan

Berdasarkan percobaan yang telah dilakukan, diperoleh hasil pengujian dari penggunaan mikrokontroler STM32 dan ESP32, konfigurasi input GPIO, debouncing, serta pembacaan sinyal analog. Analisis dari masing-masing percobaan adalah sebagai berikut.

1. **Percobaan 1: Pengenalan STM32 Blackpill (Blink)**

   Pada percobaan pertama, dilakukan pengujian program Blink pada board STM32 Blackpill menggunakan framework Arduino dan ST-Link. Program digunakan untuk mengatur kondisi LED onboard yang terhubung pada pin PC13 agar menyala dan mati secara bergantian dengan interval 500 ms. Pada board STM32 Blackpill, LED memiliki sifat aktif LOW sehingga kondisi LOW menyebabkan LED menyala, sedangkan kondisi HIGH menyebabkan LED mati.

   Percobaan ini menunjukkan penggunaan GPIO sebagai digital output dengan bantuan fungsi `pinMode()` dan `digitalWrite()`. Penggunaan framework Arduino juga mempermudah proses pemrograman STM32 karena menyediakan fungsi yang lebih sederhana dalam mengakses perangkat keras.

2. **Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)**

   Pada percobaan kedua, program Blink dijalankan menggunakan board ESP32 dengan framework ESP-IDF. Berbeda dengan framework Arduino yang menggunakan fungsi `setup()` dan `loop()`, ESP-IDF menggunakan fungsi `app_main()` sebagai titik awal program. Pengaturan GPIO dilakukan menggunakan fungsi `gpio_set_direction()` dan `gpio_set_level()`, sedangkan pengaturan waktu menggunakan `vTaskDelay()`.

Dari percobaan ini dapat diketahui bahwa framework ESP-IDF memiliki struktur pemrograman yang berbeda dengan Arduino. ESP-IDF memberikan akses yang lebih langsung terhadap fitur ESP32 dan FreeRTOS, sedangkan Arduino memiliki tingkat abstraksi yang lebih tinggi sehingga lebih mudah digunakan untuk pemrograman dasar.

3. **Percobaan 3: Pull-up dan Pull-down Eksternal vs Internal**

   Pada percobaan ketiga, dilakukan pengujian pembacaan tombol menggunakan konfigurasi pull-up dan pull-down eksternal maupun internal. Penggunaan resistor bertujuan untuk menjaga kondisi logika GPIO agar tidak mengalami floating ketika tombol tidak ditekan.

   Pada konfigurasi pull-up, pin GPIO akan membaca kondisi HIGH ketika tombol tidak ditekan dan LOW ketika tombol ditekan. Sebaliknya, pada konfigurasi pull-down, pin GPIO akan membaca kondisi LOW ketika tombol tidak ditekan dan HIGH ketika tombol ditekan.

   Penggunaan resistor eksternal maupun internal memiliki fungsi yang sama dalam menentukan kondisi logika pin. Perbedaannya terletak pada penggunaan komponen fisik, yaitu resistor eksternal yang dipasang pada breadboard atau resistor internal yang diaktifkan melalui konfigurasi program menggunakan `INPUT_PULLUP` dan `INPUT_PULLDOWN`.

4. **Percobaan 4: Debouncing pada Input Tombol GPIO**

   Pada percobaan keempat, dilakukan pengujian terhadap kondisi bouncing pada pushbutton. Bouncing merupakan perubahan sinyal HIGH dan LOW secara berulang dalam waktu singkat akibat getaran kontak mekanis ketika tombol ditekan atau dilepas. Kondisi tersebut dapat menyebabkan satu kali penekanan tombol terbaca sebagai beberapa kali input.

   Penerapan debouncing dilakukan dengan memberikan jeda waktu untuk memastikan kondisi sinyal telah stabil sebelum perubahan dianggap sebagai input yang valid. Pengujian dilakukan dengan membandingkan penghitung tanpa debouncing dan dengan debouncing. Selain itu, perubahan nilai `DEBOUNCE_DELAY` menjadi 10 ms, 50 ms, dan 200 ms digunakan untuk mengetahui pengaruh waktu penyaringan terhadap hasil pembacaan tombol.

   Semakin besar waktu debounce yang digunakan, semakin lama sistem menunggu hingga sinyal dianggap stabil. Namun, waktu yang terlalu besar dapat menyebabkan respons tombol menjadi lebih lambat. Oleh karena itu, pemilihan waktu debounce perlu disesuaikan agar pembacaan tombol tetap stabil tanpa mengurangi respons sistem secara berlebihan.

5. **Percobaan 5: Akses GPIO Analog dengan Pembacaan ADC Menggunakan LDR**

   Pada percobaan kelima, dilakukan pembacaan nilai analog menggunakan sensor LDR yang terhubung ke GPIO 34 ESP32. Berbeda dengan digital input yang hanya mengenali dua kondisi, yaitu HIGH dan LOW, analog input memungkinkan mikrokontroler membaca perubahan tegangan dalam rentang tertentu melalui ADC.

   ESP32 memiliki ADC dengan resolusi 12-bit sehingga nilai pembacaan berada pada rentang 0 hingga 4095. Nilai tersebut kemudian dikonversi menjadi perkiraan tegangan menggunakan perhitungan berdasarkan tegangan referensi 3.3V.

   Perubahan intensitas cahaya yang diterima LDR menyebabkan perubahan resistansi sensor dan tegangan pada rangkaian pembagi tegangan. Hal tersebut mengakibatkan nilai ADC yang terbaca juga mengalami perubahan. Dengan demikian, percobaan ini menunjukkan bahwa

GPIO dapat digunakan sebagai input analog untuk membaca perubahan kondisi lingkungan melalui sensor.

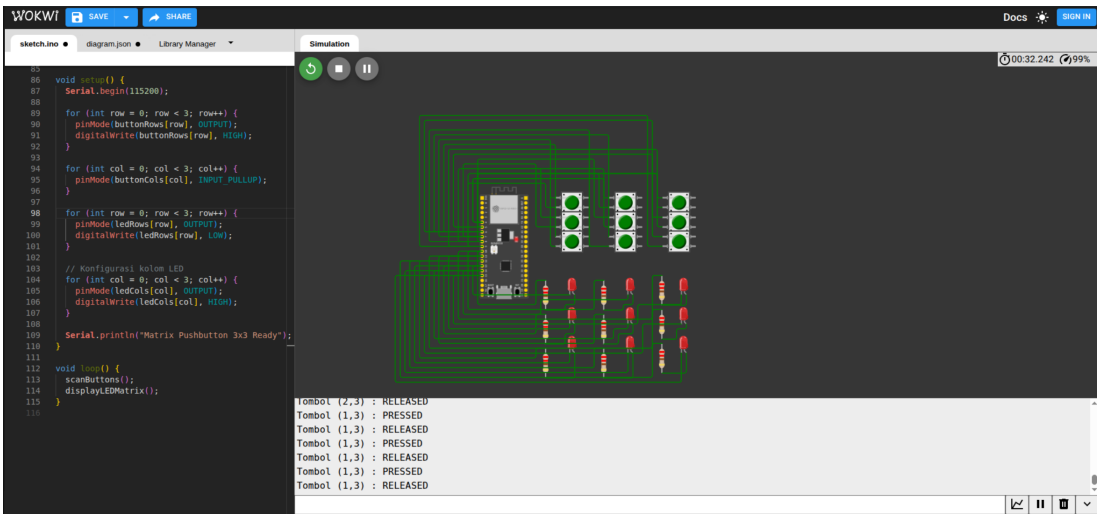
3 Hasil Tugas Modul

3.1 Simulasi Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3

Pada tugas modul ini dilakukan simulasi rangkaian Matrix Pushbutton 3x3 yang digunakan untuk mengendalikan Matrix LED 3x3 menggunakan mikrokontroler ESP32-S2. Simulasi dibuat menggunakan Wokwi dengan menyusun sembilan pushbutton dan sembilan LED dalam konfigurasi tiga baris dan tiga kolom. Setiap tombol dihubungkan berdasarkan posisi baris dan kolomnya, sehingga tombol pada posisi tertentu dapat mengendalikan LED pada posisi yang sama.

Program menggunakan metode pemindaian baris dan kolom (*matrix scanning*) untuk membaca kondisi setiap tombol. Pembacaan tombol menggunakan konfigurasi *internal pull-up* pada pin kolom, sedangkan proses pengendalian LED dilakukan dengan pemindaian matriks LED. Selain itu, program menerapkan metode *debouncing* dengan waktu stabilisasi sebesar 50 ms pada setiap tombol. Dengan metode tersebut, perubahan kondisi tombol hanya diterima apabila sinyal pembacaannya telah stabil selama waktu yang ditentukan.

Berdasarkan hasil simulasi, setiap tombol dapat mengendalikan LED sesuai dengan koordinatnya. Sebagai contoh, ketika tombol pada baris 1 kolom 1 ditekan, LED pada baris 1 kolom 1 akan menyala. Ketika tombol dilepaskan dan perubahan kondisinya telah stabil, LED pada posisi tersebut akan kembali mati. Hasil simulasi dan implementasi program ditunjukkan pada Gambar berikut.



Gambar 1: Hasil simulasi Matrix Pushbutton 3x3 dan Matrix LED 3x3 pada Wokwi

3.2 Analisis Hasil Tugas Modul

3.2.1 Penerapan Debouncing pada Matrix Pushbutton

Debouncing tetap diperlukan pada rangkaian Matrix Pushbutton 3x3 karena pushbutton merupakan komponen mekanis yang memiliki kontak fisik di dalamnya. Ketika tombol ditekan atau dilepaskan, kontak tersebut tidak selalu langsung berada pada kondisi stabil, melainkan dapat mengalami getaran atau pantulan dalam waktu singkat. Fenomena ini disebut *contact bouncing*, yang dapat menyebabk-

an sinyal keluaran berubah beberapa kali antara kondisi HIGH dan LOW meskipun tombol hanya ditekan satu kali.

Pada rangkaian matriks, kondisi tersebut dapat menyebabkan mikrokontroler salah membaca perubahan status tombol. Akibatnya, LED dapat mengalami perubahan kondisi yang tidak sesuai dengan satu kali penekanan tombol. Oleh karena itu, setiap tombol diberikan proses debouncing secara terpisah dengan waktu stabilisasi 50 ms. Perubahan status tombol baru diterima setelah sinyal pembacaan berada pada kondisi yang sama selama waktu tersebut. Dengan demikian, LED tidak langsung mengikuti perubahan sinyal mentah yang masih mengalami bouncing, tetapi hanya berubah setelah kondisi tombol benar-benar stabil.

3.2.2 Konfigurasi Pull-up pada Pembacaan Tombol

Pada rangkaian ini digunakan konfigurasi *internal pull-up* dengan mengatur pin kolom tombol menggunakan fungsi `INPUT_PULLUP`. Konfigurasi tersebut membuat pin GPIO memiliki hubungan internal menuju tegangan logika HIGH melalui resistor pull-up. Ketika tombol tidak ditekan, pin kolom akan terbaca HIGH karena tidak terhubung dengan jalur GND. Sebaliknya, ketika tombol ditekan pada baris yang sedang dipindai, pin kolom akan terhubung ke GND sehingga terbaca LOW.

Kondisi pin tidak boleh dibiarkan *floating* karena pin GPIO tidak memiliki level tegangan yang pasti ketika tidak terhubung ke HIGH maupun LOW. Hal tersebut dapat menyebabkan pembacaan berubah-ubah akibat gangguan listrik dari lingkungan sekitar. Akibatnya, mikrokontroler dapat menganggap tombol sedang ditekan meskipun sebenarnya tidak ada penekanan. Penggunaan pull-up memberikan kondisi logika yang pasti sehingga pembacaan tombol menjadi lebih stabil dan dapat diandalkan.

3.2.3 Perbedaan Digital Input dan Analog Input

Pembacaan digital input dan analog input memiliki perbedaan pada jenis informasi yang dihasilkan. Pada digital input, mikrokontroler hanya mengenali dua kondisi logika, yaitu HIGH dan LOW. Dalam rangkaian pushbutton, kedua kondisi tersebut digunakan untuk menentukan apakah tombol sedang ditekan atau tidak. Nilai digital diperoleh dengan membandingkan tegangan masukan terhadap batas logika yang ditentukan oleh mikrokontroler.

Sementara itu, analog input menggunakan *Analog to Digital Converter* (ADC) untuk mengubah tegangan analog menjadi nilai digital. Berbeda dengan digital input, ADC dapat menghasilkan banyak variasi nilai sesuai dengan tegangan yang diterima. Pada ESP32-S2 dengan resolusi ADC 12-bit, hasil konversi secara teoritis berada pada rentang 0 sampai 4095. Hal ini memungkinkan perubahan tegangan yang dihasilkan oleh sensor LDR akibat perbedaan intensitas cahaya untuk direpresentasikan dalam bentuk nilai yang lebih beragam.

Apabila salah satu tombol pada rangkaian digantikan dengan LDR, pembacaan tidak lagi hanya menunjukkan kondisi ditekan atau tidak ditekan. Nilai ADC dapat digunakan untuk mengetahui perubahan intensitas cahaya yang diterima sensor. Semakin besar perubahan tegangan pada rangkaian pembagi tegangan LDR, semakin berbeda pula nilai hasil konversi ADC yang terbaca oleh mikrokontroler. Dengan demikian, digital input lebih sesuai untuk mendeteksi dua kondisi, sedangkan analog input dapat digunakan untuk mengukur perubahan besaran fisik secara bertingkat.

4 Kesimpulan

Berdasarkan praktikum Modul 1 yang telah dilaksanakan, dapat disimpulkan bahwa PlatformIO pada Visual Studio Code dapat digunakan sebagai lingkungan pengembangan program untuk mikrokontroler STM32 dan ESP32. Praktikan juga dapat memahami perbedaan penggunaan framework Arduino dan ESP-IDF, baik dari segi struktur program maupun fungsi yang digunakan dalam mengakses GPIO.

Selain itu, praktikan dapat memahami konfigurasi pull-up dan pull-down secara eksternal maupun internal untuk menjaga kestabilan kondisi input GPIO. Penerapan debouncing juga diperlukan untuk mengatasi perubahan sinyal akibat bouncing pada tombol mekanis. Pada percobaan terakhir, penggunaan ADC pada ESP32 memungkinkan pembacaan perubahan tegangan dari sensor LDR berdasarkan intensitas cahaya yang diterima.

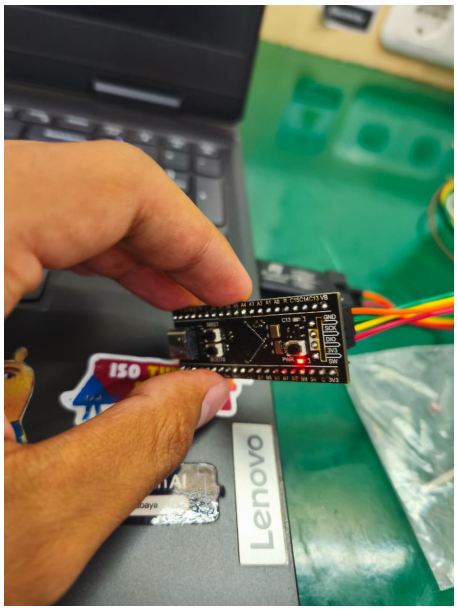
Secara keseluruhan, praktikum ini memberikan pemahaman mengenai penggunaan GPIO sebagai digital output, digital input, dan analog input, serta penerapannya dalam pemrograman sistem tertanam menggunakan STM32 dan ESP32.

5 Lampiran

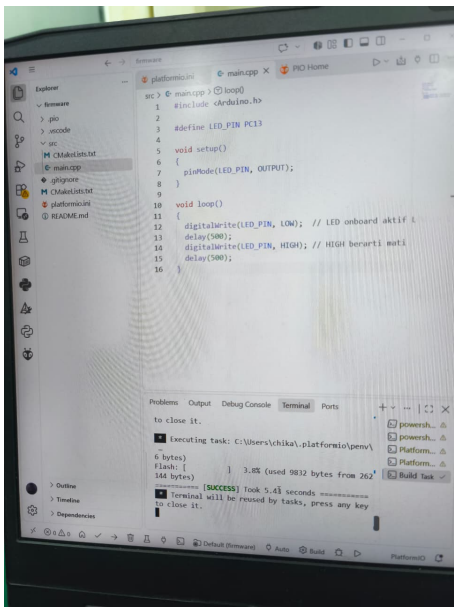
6 Lampiran

6.1 Dokumentasi Percobaan 1

Dokumentasi pelaksanaan Percobaan 1 menggunakan STM32 Blackpill.



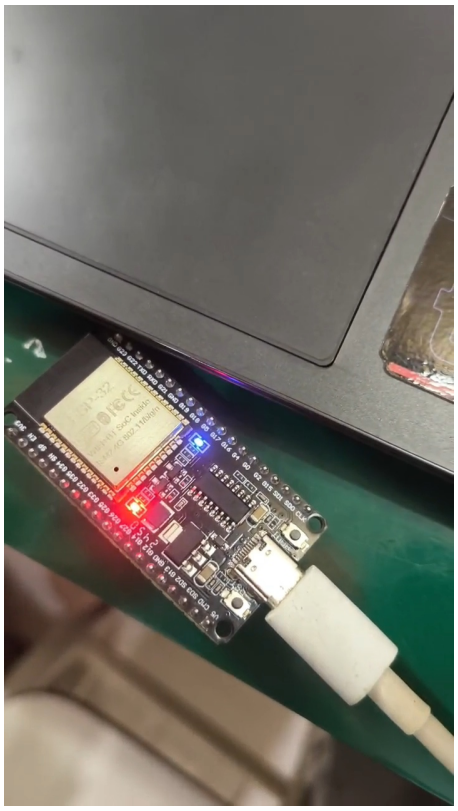
Gambar 2: Dokumentasi rangkaian Percobaan 1



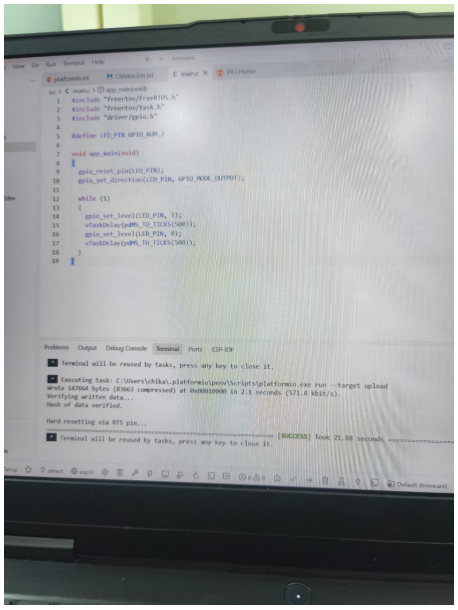
Gambar 3: Dokumentasi kode Percobaan 1

6.2 Dokumentasi Percobaan 2

Dokumentasi pelaksanaan Percobaan 2 menggunakan ESP32 dengan ESP-IDF.



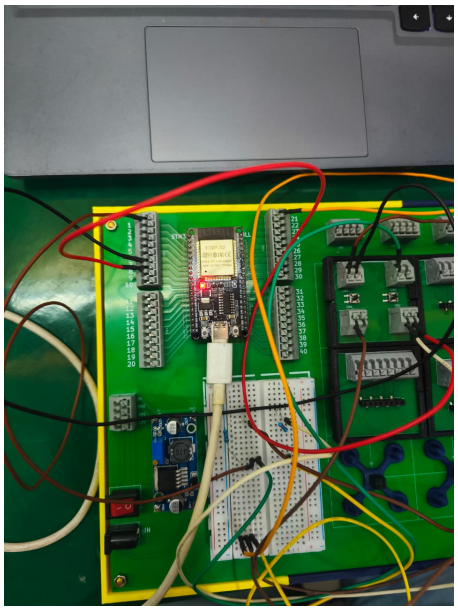
Gambar 4: Dokumentasi rangkaian Percobaan 2



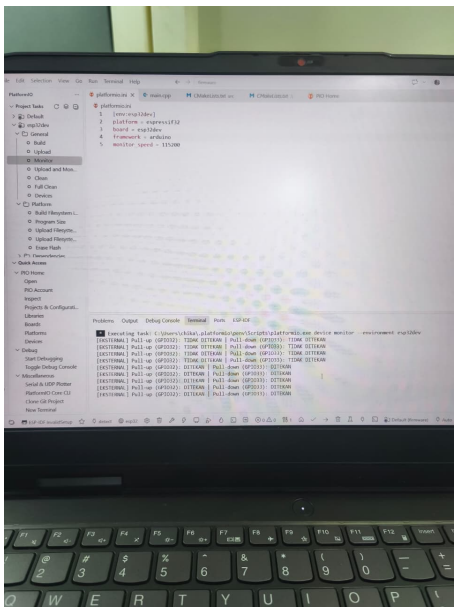
Gambar 5: Dokumentasi kode Percobaan 2

6.3 Dokumentasi Percobaan 3

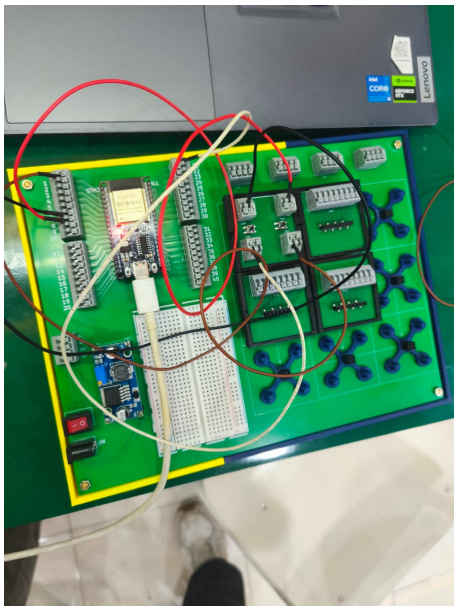
Dokumentasi pelaksanaan Percobaan 3 mengenai konfigurasi pull-up dan pull-down pada pushbutton.



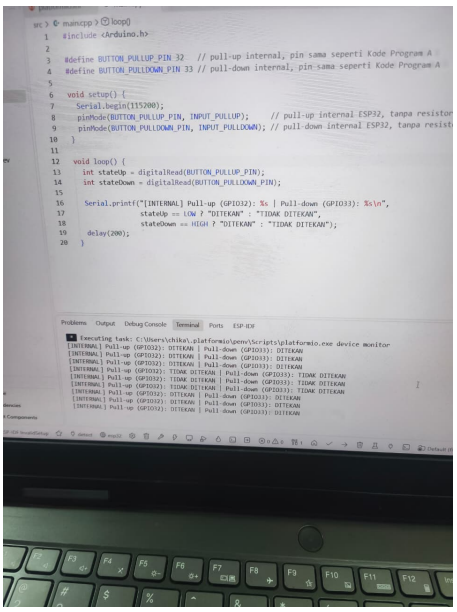
Gambar 6: Dokumentasi rangkaian Percobaan 3



Gambar 7: Hasil pengamatan Percobaan 3a



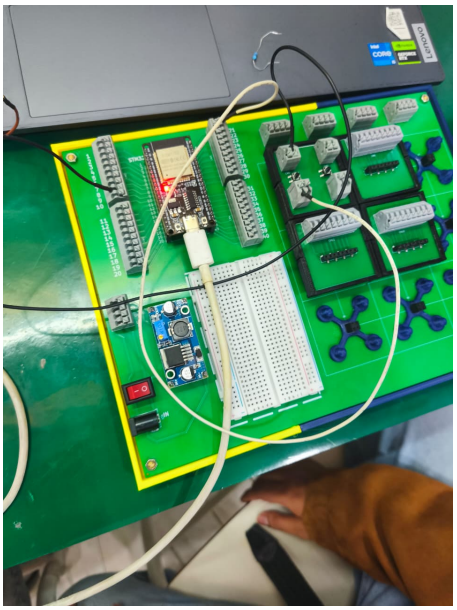
Gambar 8: Dokumentasi rangkaian Percobaan 3b



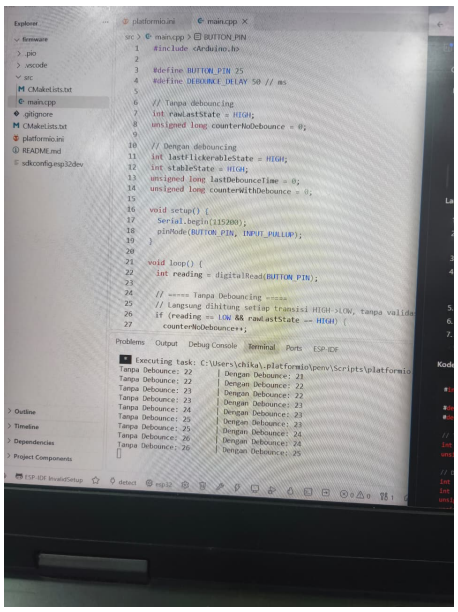
Gambar 9: Hasil pengamatan Percobaan 3b

6.4 Dokumentasi Percobaan 4

Dokumentasi pelaksanaan Percobaan 4 mengenai debouncing pada pushbutton.



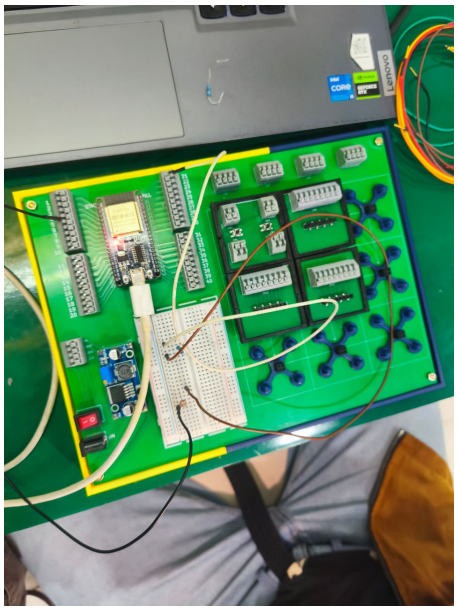
Gambar 10: Dokumentasi rangkaian Percobaan 4



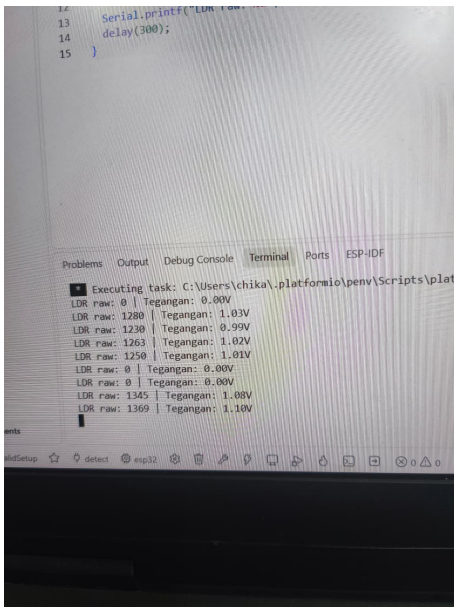
Gambar 11: Hasil pengamatan Percobaan 4 menggunakan Serial Plotter

6.5 Dokumentasi Percobaan 5

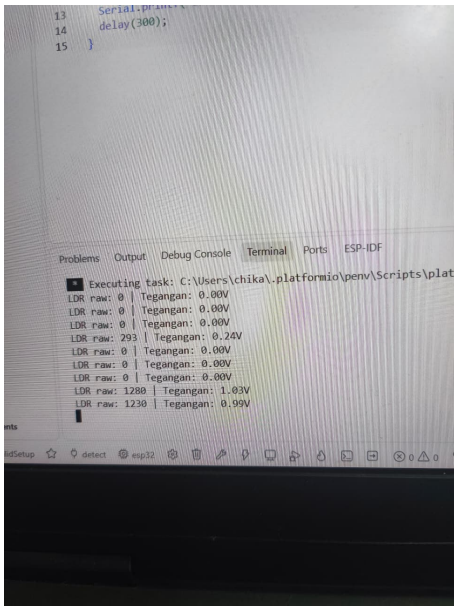
Dokumentasi pelaksanaan Percobaan 5 mengenai pembacaan sensor LDR menggunakan ADC pada ESP32.



Gambar 12: Dokumentasi rangkaian Percobaan 5



Gambar 13: Hasil pembacaan sensor LDR pada Serial Plotter



Gambar 14: Dokumentasi hasil pengamatan lanjutan Percobaan 5